

Codificação da Informação

Como textos, números e imagens são transformados em bits para processamento e armazenamento.



01000001

A

símbolo → código → bits

Roteiro da aula

O percurso começa no conceito de codificação e avança até exemplos práticos de texto e imagem.

1 Codificação
e bits

2 Tamanhos
binários

3 Números

4 Caracteres
de texto

5 Imagens
digitais

6 Dinâmica
perguntas

01000001 01100010 00110011 11100010 10000010 10101100

Codificar é traduzir representações

O usuário vê símbolos; o computador armazena e processa padrões binários.

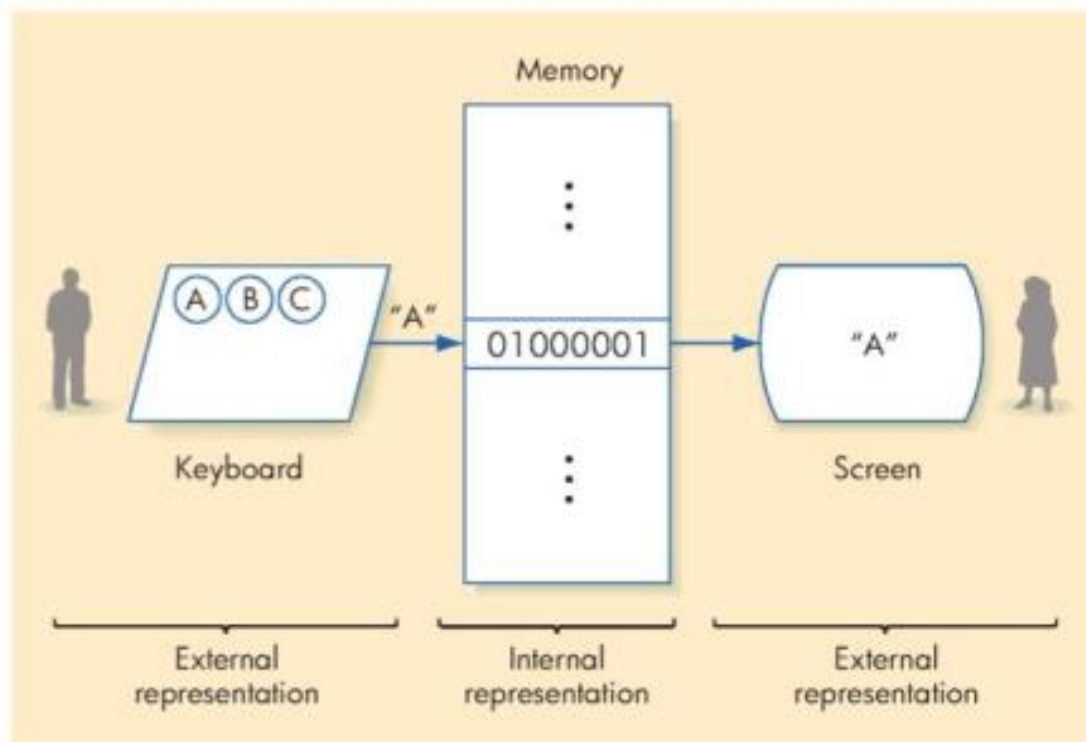


Figura original: representação externa → interna → externa

Codificação associa significados a sequências de bits.

A mesma informação pode ter diferentes formatos de armazenamento.

Decodificar é recuperar a representação compreensível para humanos.



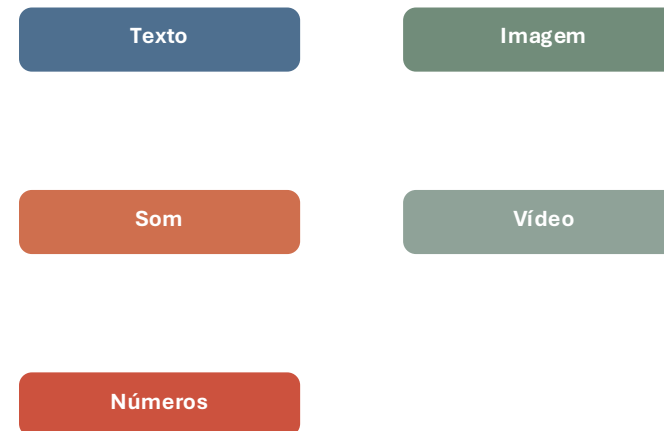
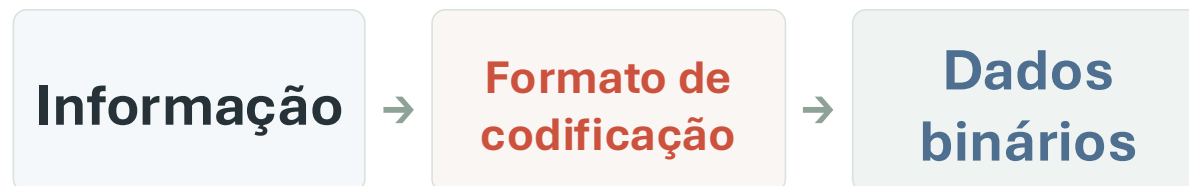
Tudo vira dados binários

Números, caracteres, som, imagem e vídeo usam formatos específicos de codificação.

Na computação atual, a representação interna é binária.

Cada tipo de informação exige regras próprias de codificação.

O formato escolhido afeta tamanho, precisão, compatibilidade e processamento.



**Um bit não “sabe” o que representa.
Quem dá sentido aos bits é a
codificação.**

De onde viemos: Boole

A álgebra de Boole conecta lógica, matemática e chaveamentos.

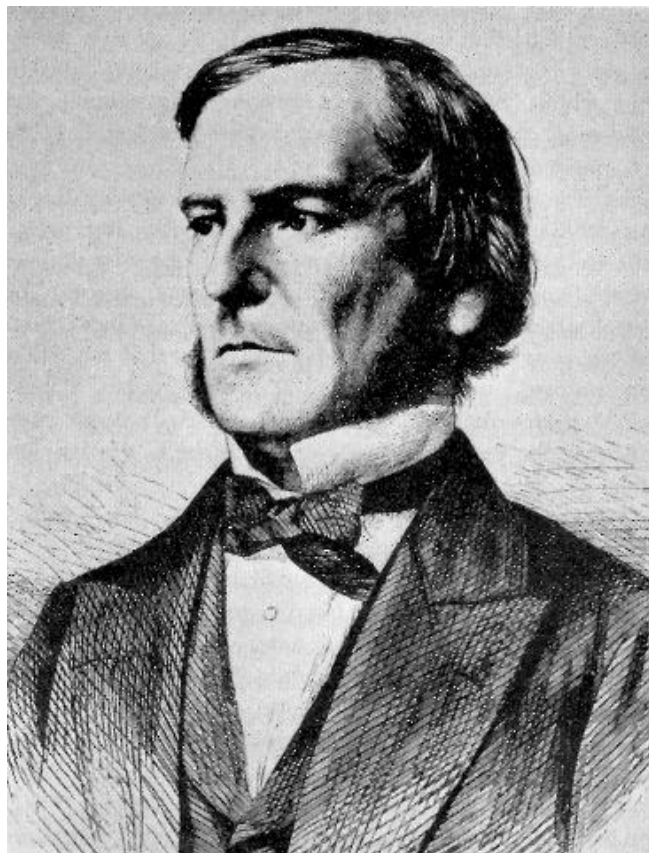
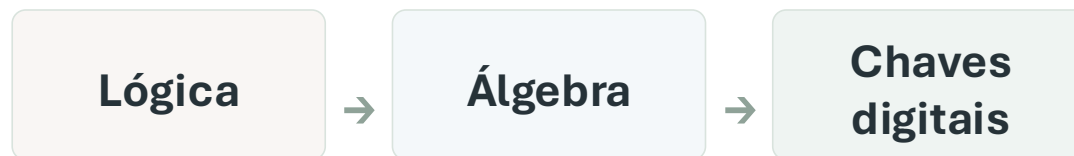


Figura original: George Boole (1815-1864)

Estados lógicos podem ser tratados formalmente: verdadeiro/falso.

Em circuitos, esses estados aparecem como ligado/desligado ou 1/0.

Essa ponte entre lógica e hardware fundamenta a computação digital.



De onde viemos: Shannon

Computadores digitais são circuitos eletrônicos digitais compostos por portas lógicas.

Portas lógicas implementam operações como E, OU e NÃO.

A informação é manipulada por sinais digitais representados por bits.

Codificar é necessário para colocar dados reais dentro desse modelo binário.

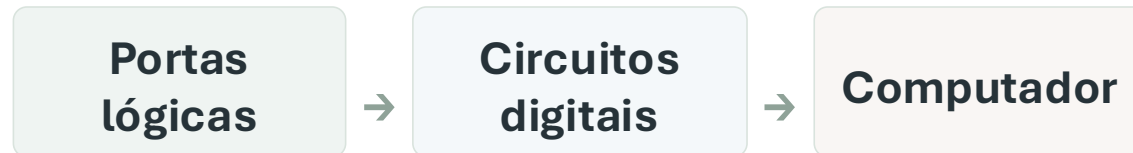


Figura original: Claude Shannon (1916-2001)

Bit e byte

As unidades básicas permitem contar armazenamento, memória e transmissão.

bit

menor unidade de informação
valor possível: 0 ou 1



8 posições binárias formam 1 byte

byte

agrupamento usual
1 byte = 8 bits

Com 8 bits, temos $2^8 = 256$ combinações possíveis.

Tamanhos binários: prefixos

Nem todo “quilo” em computação significa exatamente a mesma coisa.

Factor	Name	Symbol	Origin	Derivation
2^{10}	kibi	Ki	kilobinary: $(2^{10})^1$	kilo: $(10^3)^1$
2^{20}	mebi	Mi	megabinary: $(2^{10})^2$	mega: $(10^3)^2$
2^{30}	gibi	Gi	gigabinary: $(2^{10})^3$	giga: $(10^3)^3$
2^{40}	tebi	Ti	terabinary: $(2^{10})^4$	tera: $(10^3)^4$
2^{50}	pebi	Pi	petabinary: $(2^{10})^5$	peta: $(10^3)^5$
2^{60}	exbi	Ei	exabinary: $(2^{10})^6$	exa: $(10^3)^6$

Factor	Name	Symbol
10^{24}	yotta	Y
10^{21}	zetta	Z
10^{18}	exa	E
10^{15}	peta	P
10^{12}	tera	T
10^9	giga	G
10^6	mega	M
10^3	kilo	k

kilo (k) = 10^3 ; kibi (Ki) = 2^{10} .
mega (M) = 10^6 ; mebi (Mi) = 2^{20} .
A diferença cresce conforme a unidade aumenta.

Tabelas originais: prefixos binários e decimais

one kibibit	1 Kibit = 2^{10} bit = 1024 bit
one kilobit	1 kbit = 10^3 bit = 1000 bit
one mebibyte	1 MiB = 2^{20} B = 1 048 576 B
one megabyte	1 MB = 10^6 B = 1 000 000 B
one gibibyte	1 GiB = 2^{30} B = 1 073 741 824 B
one gigabyte	1 GB = 10^9 B = 1 000 000 000 B

KB não é KiB

A distinção evita confusão entre armazenamento, memória e especificações técnicas.

Símbolo	Base	Valor	Uso comum
1 KB	10^3	1.000 bytes	armazenamento decimal
1 KiB	2^{10}	1.024 bytes	memória/sistemas
1 MB	10^6	1.000.000 bytes	fabricantes/discos
1 MiB	2^{20}	1.048.576 bytes	sistemas operacionais

Regra prática

Quando a precisão importa, informe a unidade explicitamente: MB ou MiB, GB ou GiB.

Números também têm formatos

Representações numéricas mudam conforme o conjunto de valores e as operações desejadas.

Naturais: binário simples.

Inteiros: complemento de dois, sinal-magnitude ou excesso.

Reais: representações de ponto flutuante, como IEEE 754.

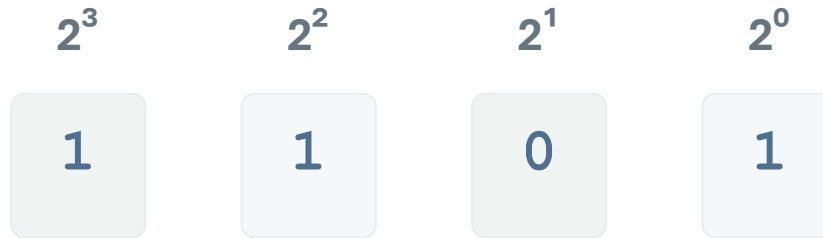
Cada escolha tem limites e comportamento próprio.



Mesmo “número” pode exigir codificações diferentes.

Naturais: binário simples

Cada posição representa uma potência de 2.



$$= 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 13$$

A leitura é posicional.

Cada bit à esquerda vale o dobro do bit anterior.

Com n bits sem sinal, o intervalo é de 0 a $2^n - 1$.

Inteiros: complemento de dois

Formato amplamente usado para representar valores positivos e negativos.

O bit mais significativo contribui com peso negativo.

Operações de soma/subtração ficam simples para o hardware.

Com n bits, o intervalo é de -2^{n-1} a $2^{n-1} - 1$.

Exemplo com 4 bits

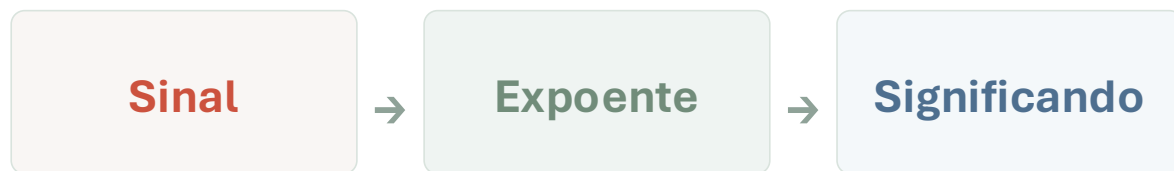
1011_2

$$\begin{aligned} &= -8 + 0 + 2 + 1 \\ &= -5 \end{aligned}$$

4 bits: -8 a +7

Reais: ponto flutuante

Números reais são aproximados por sinal, expoente e significando.

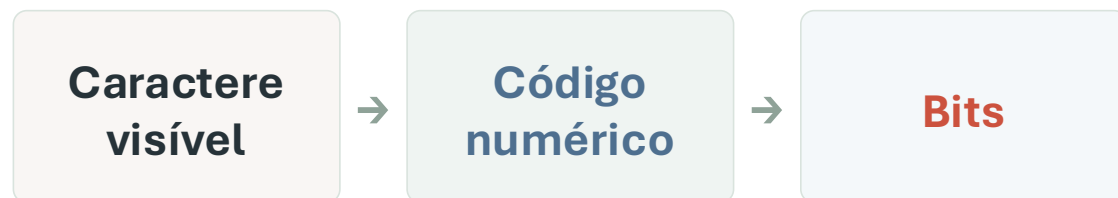


$$\text{valor} \approx (-1)^s \times \text{significando} \times 2^{\text{expoente}}$$

Permite representar números muito pequenos e muito grandes. Nem todo decimal tem representação binária finita. Arredondamento e perda de precisão fazem parte do modelo.

Caracteres de texto

Textos precisam mapear símbolos para códigos numéricos.



ASCII: importante historicamente e ainda presente por compatibilidade.
Unicode: padrão dominante para sistemas modernos e múltiplas línguas.
Codificações como UTF-8 definem como o código será armazenado em bytes.

"A" → 65 → 01000001

ASCII: um código histórico

American Standard Code for Information Interchange.

Código numérico para caracteres visuais e de controle.

Originalmente: 7 bits, com 128 combinações.

Extended ASCII: 8 bits, com 256 combinações.

Foi substituído em grande parte pelo Unicode, mas continua essencial para compatibilidade.

ASCII

7 bits

$2^7 = 128$ códigos

Tabela ASCII

Códigos de controle e caracteres imprimíveis em decimal, hexadecimal e octal.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.asciitable.com

Tabela original: códigos ASCII. Fonte indicada na imagem: asciitable.com

Extended ASCII

A extensão para 8 bits ampliou o número de caracteres, mas não resolveu a diversidade linguística.

128	Ç	144	É	160	á	176	☐	193	⌞	209	ƒ	225	ß	241	±
129	ü	145	æ	161	í	177	☐	194	⌞	210	π	226	Γ	242	≥
130	é	146	Æ	162	ó	178	☐	195	⌞	211	ℓ	227	π	243	≤
131	â	147	ô	163	ú	179		196	—	212	ℓ	228	Σ	244	ƒ
132	ä	148	ö	164	ñ	180	†	197	†	213	ƒ	229	σ	245	∫
133	à	149	ò	165	Ñ	181	†	198	†	214	π	230	μ	246	÷
134	â	150	û	166	°	182		199		215	†	231	τ	247	≈
135	ç	151	ù	167	°	183	π	200	ℓ	216	≠	232	Φ	248	°
136	ê	152	—	168	¿	184	¶	201	ƒ	217	∫	233	Θ	249	.
137	ë	153	Ö	169	—	185	¶	202	⌞	218	∫	234	Ω	250	.
138	è	154	Û	170	¬	186		203	π	219	■	235	δ	251	√
139	ï	156	£	171	½	187	¶	204	†	220	■	236	∞	252	—
140	î	157	¥	172	¼	188	¶	205	=	221	■	237	φ	253	²
141	ï	158	—	173	¡	189	¶	206	†	222	■	238	ε	254	■
142	Ä	159	ƒ	174	«	190	¶	207	⌞	223	■	239	∩	255	
143	Å	192	ℓ	175	»	191	¶	208	⌞	224	α	240	≡		

Source: www.asciitable.com

8 bits → 256 valores possíveis.
Tabelas estendidas variavam conforme idioma, país e plataforma.
Problema central: interoperabilidade limitada.

Tabela original: códigos de 128 a 255. Fonte indicada na imagem: asciitable.com

Unicode: um número para cada caractere

A proposta é separar o caractere de detalhes de plataforma, programa ou língua.



Logo presente no material original

Unicode fornece um número único para cada caractere.

Independente de plataforma.
Independente de programa.
Independente de língua.
Inclui letras, dígitos, símbolos e caracteres de controle.

Ponto de código

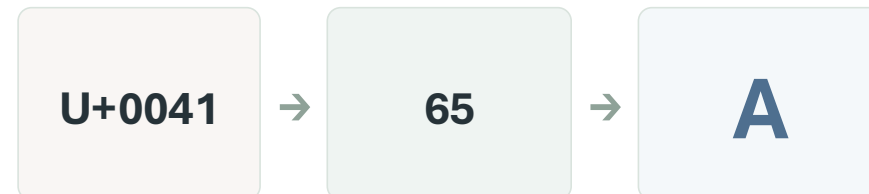
No Unicode, cada caractere recebe um identificador como U+0041.

U+0041

hexadecimal 0041

decimal 65

caractere: LATIN CAPITAL
LETTER A



**Ponto de código ≠ bytes gravados em
arquivo.**

A forma de codificação define esses bytes.

Formas de codificação Unicode

UTF-8, UTF-16 e UTF-32 armazenam pontos de código de maneiras diferentes.

Codificação	Tamanho	Observação
UTF-8	variável	ASCII mantém o mesmo valor; muito usado na web
UTF-16	1 ou 2 unidades de 16 bits	comum em algumas APIs e plataformas
UTF-32	fixo em 32 bits	simples de indexar, mas ocupa mais espaço

Ideia-chave

**Unicode define os caracteres.
UTF define como eles viram bytes.**

Visualizando blocos Unicode

Tabelas organizam os pontos de código e seus respectivos caracteres.

	000	001	002	003	004	005	006	007
0	NUL 0000	DLE 0010	SP 0020	0 0030	@ 0040	P 0050	` 0060	p 0070
1	SOH 0001	DC1 0011	! 0021	1 0031	A 0041	Q 0051	a 0061	q 0071
2	STX 0002	DC2 0012	" 0022	2 0032	B 0042	R 0052	b 0062	r 0072
3	ETX 0003	DC3 0013	# 0023	3 0033	C 0043	S 0053	c 0063	s 0073
4	EOT 0004	DC4 0014	\$ 0024	4 0034	D 0044	T 0054	d 0064	t 0074
5	ENQ 0005	NAK 0015	% 0025	5 0035	E 0045	U 0055	e 0065	u 0075

Tabela presente no material original: amostra de pontos de código Unicode

O caractere é identificado por posição na tabela.

O mesmo padrão organiza letras, números, símbolos e controles.

Sistemas diferentes podem exibir fontes diferentes, mas o código é o mesmo.

Imagens: bitmap e vetorial

Dois modelos principais para representar imagens digitalmente.

Bitmap / Raster

Matriz de pixels.
Cada pixel armazena cor e propriedades.
Pode perder qualidade ao ampliar ou compactar.

Vetorial

Estruturas matemáticas e geométricas.
Escala sem perda de qualidade.
Muito usado em logos, ícones e ilustrações.

Escolher o modelo certo depende do tipo de imagem e do uso esperado.

Bitmap: imagem como matriz

Ao ampliar uma imagem raster, a estrutura em pixels fica visível.

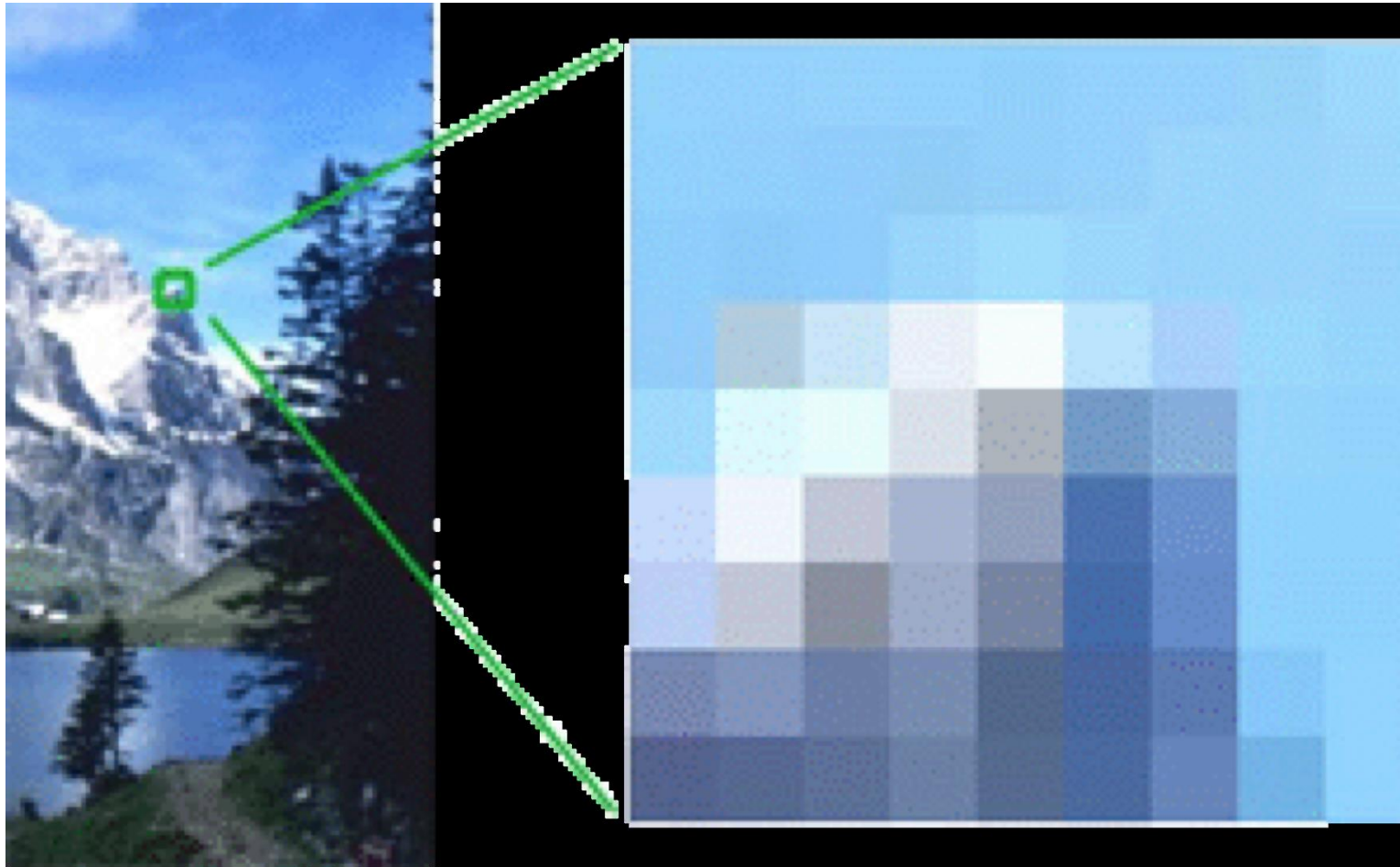


Figura original: detalhe ampliado evidencia os pixels

Pixel = Picture Element.
Resolução indica quantos pixels compõem a imagem. Mais pixels podem significar mais detalhes e maior tamanho de arquivo.

Tipos de bitmap e cor

A quantidade de bits por pixel influencia a paleta de cores possível.

Tipo	Característica	Bits por pixel
Line-art	duas cores, usualmente preto e branco	1 bit
Grayscale	diversos tons de cinza	8 bits ou mais
Colorida	sistemas de cores como RGB ou CMYK	vários bits

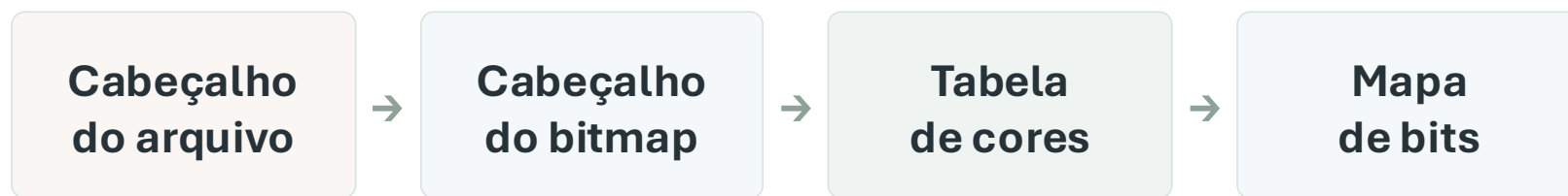
RGB

**Cada canal
representa uma
intensidade de cor.**

Quanto mais bits por pixel, maior a capacidade de representar variações de cor.

Estrutura BMP do Windows

Um arquivo de imagem contém metadados e dados dos pixels.



Cabeçalhos descrevem dimensões, profundidade de cor e organização.

Tabela de cores pode ser usada em imagens indexadas.

Mapa de bits contém os valores dos pixels propriamente ditos.

.BMP

arquivo =
estrutura +
dados

Para pensar...

O limite prático não é apenas representar: é armazenar, transmitir e processar em escala.

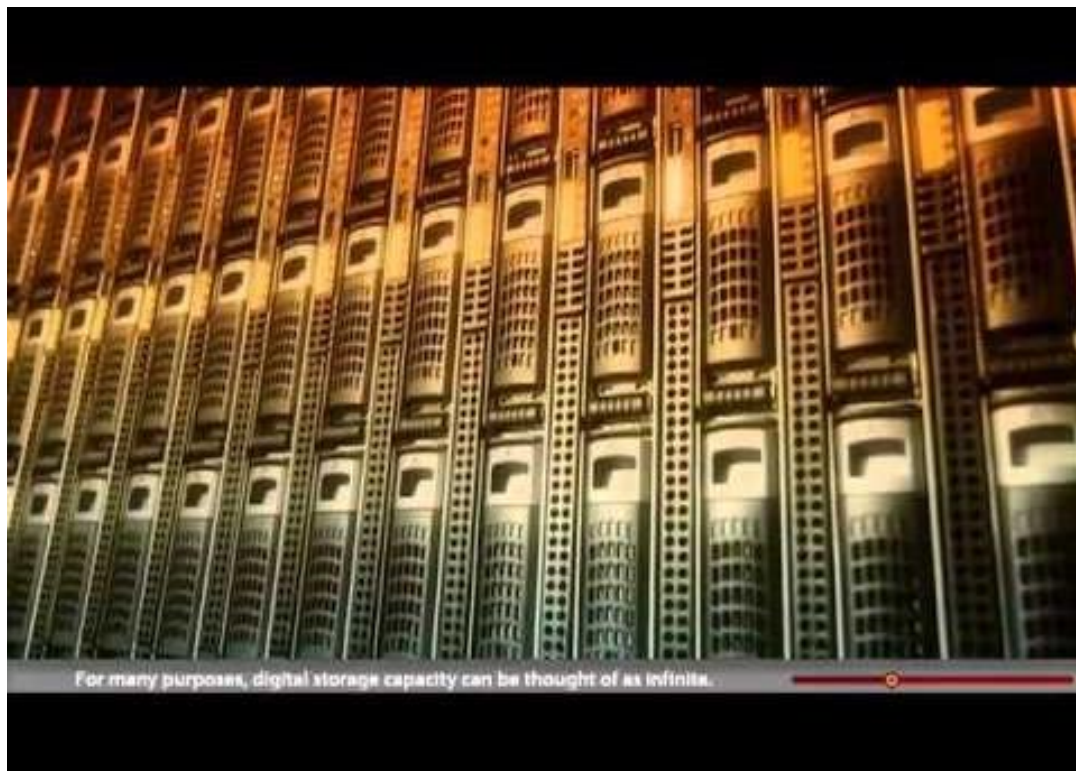


Imagem presente no material original: armazenamento digital em grande escala

Quanto espaço ocupa uma imagem sem compactação?

O que muda quando usamos compactação?

Como compatibilidade e padrão afetam a preservação de dados?

Por que codificações antigas ainda aparecem em sistemas reais?

Pergunta 1

Dinâmica de revisão: responda antes de avançar.



O que é um bit?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 1

Use a resposta para fechar a discussão e conectar com o próximo conceito.



Bit é a menor unidade de informação em computação. Ele pode assumir apenas dois valores: 0 ou 1.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 2

Dinâmica de revisão: responda antes de avançar.



Por que computadores digitais utilizam representação binária?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 2

Use a resposta para fechar a discussão e conectar com o próximo conceito.



Porque circuitos digitais trabalham naturalmente com dois estados físicos estáveis, associados a 0 e 1.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 3

Dinâmica de revisão: responda antes de avançar.



Quantos bits existem em 1 byte?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 3

Use a resposta para fechar a discussão e conectar com o próximo conceito.



1 byte possui 8 bits. Por isso, 1 byte pode representar $2^8 = 256$ combinações.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 4

Dinâmica de revisão: responda antes de avançar.



Qual é a diferença entre KB e KiB?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 4

Use a resposta para fechar a discussão e conectar com o próximo conceito.



KB usa base decimal: 1 KB = 1.000 bytes. KiB usa base binária: 1 KiB = 1.024 bytes.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 5

Dinâmica de revisão: responda antes de avançar.



Quantos caracteres o ASCII original de 7 bits consegue representar?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 5

Use a resposta para fechar a discussão e conectar com o próximo conceito.



O ASCII original representa $2^7 = 128$ códigos diferentes.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 6

Dinâmica de revisão: responda antes de avançar.



Por que o Extended ASCII não foi suficiente para sistemas modernos?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 6

Use a resposta para fechar a discussão e conectar com o próximo conceito.



Porque 256 códigos não bastam para representar, de forma padronizada, caracteres de muitas línguas e símbolos.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 7

Dinâmica de revisão: responda antes de avançar.



O que significa o ponto de código Unicode U+0041?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 7

Use a resposta para fechar a discussão e conectar com o próximo conceito.



U+0041 é o ponto de código do caractere “A” maiúsculo latino; em decimal, corresponde ao valor 65.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 8

Dinâmica de revisão: responda antes de avançar.



Qual é a diferença essencial entre UTF-8 e UTF-32?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 8

Use a resposta para fechar a discussão e conectar com o próximo conceito.



**UTF-8 usa quantidade variável de bytes por caractere.
UTF-32 usa tamanho fixo de 32 bits por ponto de código.**

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 9

Dinâmica de revisão: responda antes de avançar.



Qual é a principal diferença entre imagem bitmap e imagem vetorial?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 9

Use a resposta para fechar a discussão e conectar com o próximo conceito.



Bitmap é formado por pixels em uma matriz. Vetorial é definido por formas geométricas e equações.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.

Pergunta 10

Dinâmica de revisão: responda antes de avançar.



Por que uma imagem bitmap pode ficar “pixelada” ao ser ampliada?

Discuta em duplas por 1 minuto. Depois, avance para comparar com a resposta.

01000001 01100010 00110011 11100010 10000010 10101100

Resposta 10

Use a resposta para fechar a discussão e conectar com o próximo conceito.



Porque a ampliação torna visíveis os pixels da matriz original; se não há pixels suficientes, a imagem perde nitidez.

Ponto de reforço para o professor: peça um exemplo adicional aos alunos antes de seguir.